

INFOE 3

LAB 2: HASHING

Francisco, Maica Merylle

Manulat, Faith Joy

Cortejo, Kevin Edyson

Post Questions:

1. In this specific activity, which one is the Subject (the entity that acts to conduct the check) and which is the Object (the entity that is the target of the check)?

Answer:

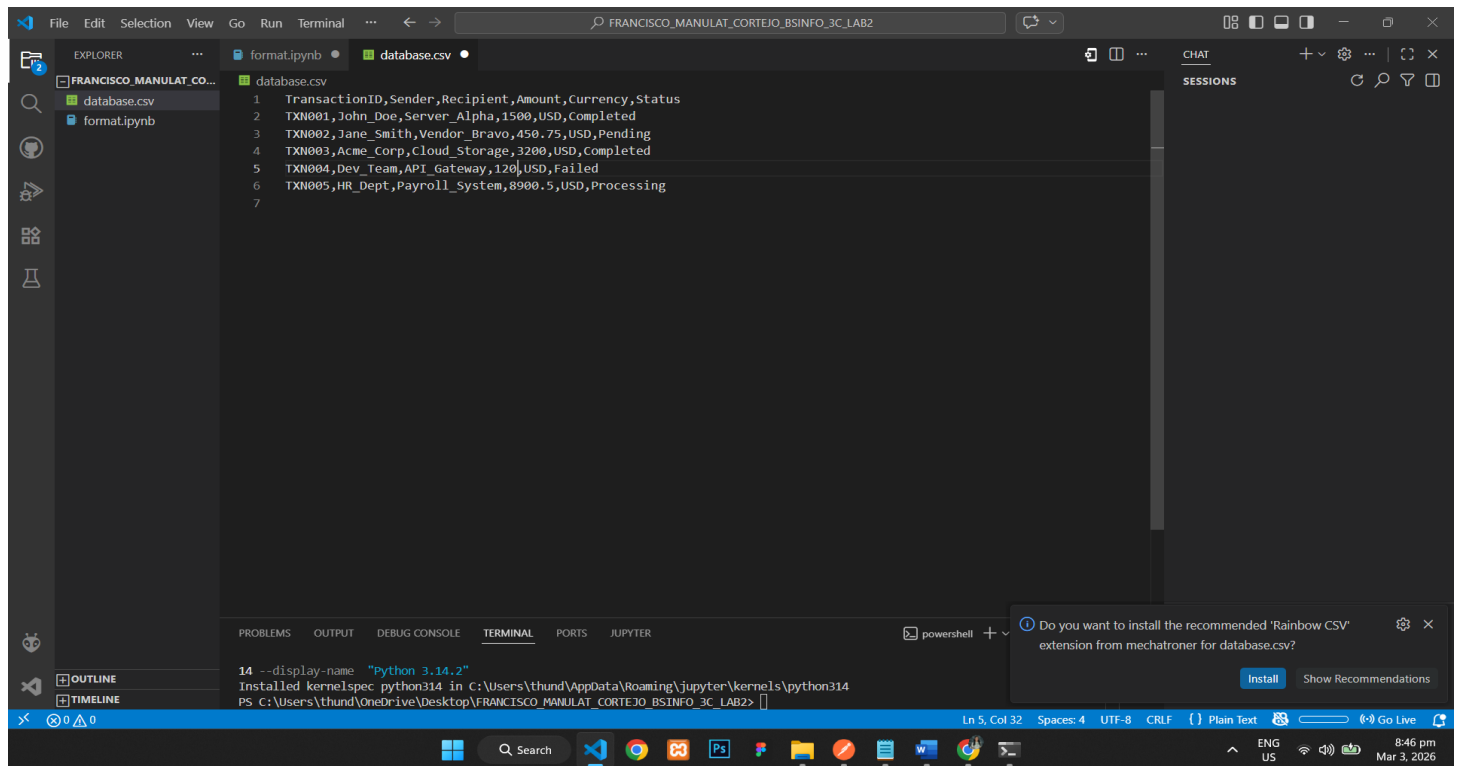
- In this specific activity, the **subject** is the user or the running program because it is the entity that performs the action of checking or modifying the file, while the **object** is the file or system resource itself since it is the target being accessed, checked, or changed.

2. If you manually open the file and change one variable (from Instructions), is that action considered a Vulnerability or a Threat Event?

Answer:

- If you manually open the file and change one variable from the instructions, that action is considered a **threat event** because it represents an actual occurrence where a system resource is modified, potentially exploiting an existing vulnerability such as weak access controls or lack of protection.

Screenshots:



```
1 TransactionID,Sender,Recipient,Amount,Currency,Status
2 TXN001,John_Doe,Server_Alpha,1500,USD,Completed
3 TXN002,Jane_Smith,Vendor_Bravo,450.75,USD,Pending
4 TXN003,Acme_Corp,Cloud_Storage,3200,USD,Completed
5 TXN004,Dev_Team,API_Gateway,120,USD,Failed
6 TXN005,HR_Dept,Payroll_System,8900.5,USD,Processing
7
```

14 --display-name "Python 3.14.2"
Installed kernelspec python314 in C:\Users\thund\AppData\Roaming\jupyter\kernels\python314
PS C:\Users\thund\OneDrive\Desktop\FRANCISCO_MANULAT_CORTEJO_BSINFO_3C_LAB2>

Here is the original unmodified database.csv

INFOE 3

LAB 2: HASHING

Francisco, Maica Merylle

Manulat, Faith Joy

Cortejo, Kevin Edyson

```
1 TransactionID,Sender,Recipient,Amount,Currency,Status
2 TXN001,John_Doe,Server_Alpha,1500,USD,Completed
3 TXN002,Jane_Smith,Vendor_Bravo,450.75,USD,Pending
4 TXN003,Acme_Corp,Cloud_Storage,3200,USD,Completed
5 TXN004,Dev_Team,API_Gateway,1200,USD,Failed
6 TXN005,HR_Dept,Payroll_System,8900.5,USD,Processing
7
```

Here is the manually modified database.csv

```
import hashlib

filename = 'database.csv'

def get_file_hash(path):
    sha256_hash = hashlib.sha256()
    with open(path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()

# Generate and print the result
try:
    print(f"File: {filename}")
    print(f"SHA-256 Hash: {get_file_hash(filename)}") # copy the hash and paste it into 'EXPECTED_HASH' variable.
    print("-" * 30)
except FileNotFoundError:
    print(f"Error: '{filename}' not found.")
```

File: database.csv
SHA-256 Hash: a83ae7f10e647eba3edf9d899fcb32b1e450fcb5194365c4a93c470fd4b6c86d

This screenshot shows the execution of the first cell, where the original SHA-256 hash of the **database.csv** file is generated. This hash serves as a unique “digital fingerprint” that establishes the baseline for the file’s integrity before any modifications are made.

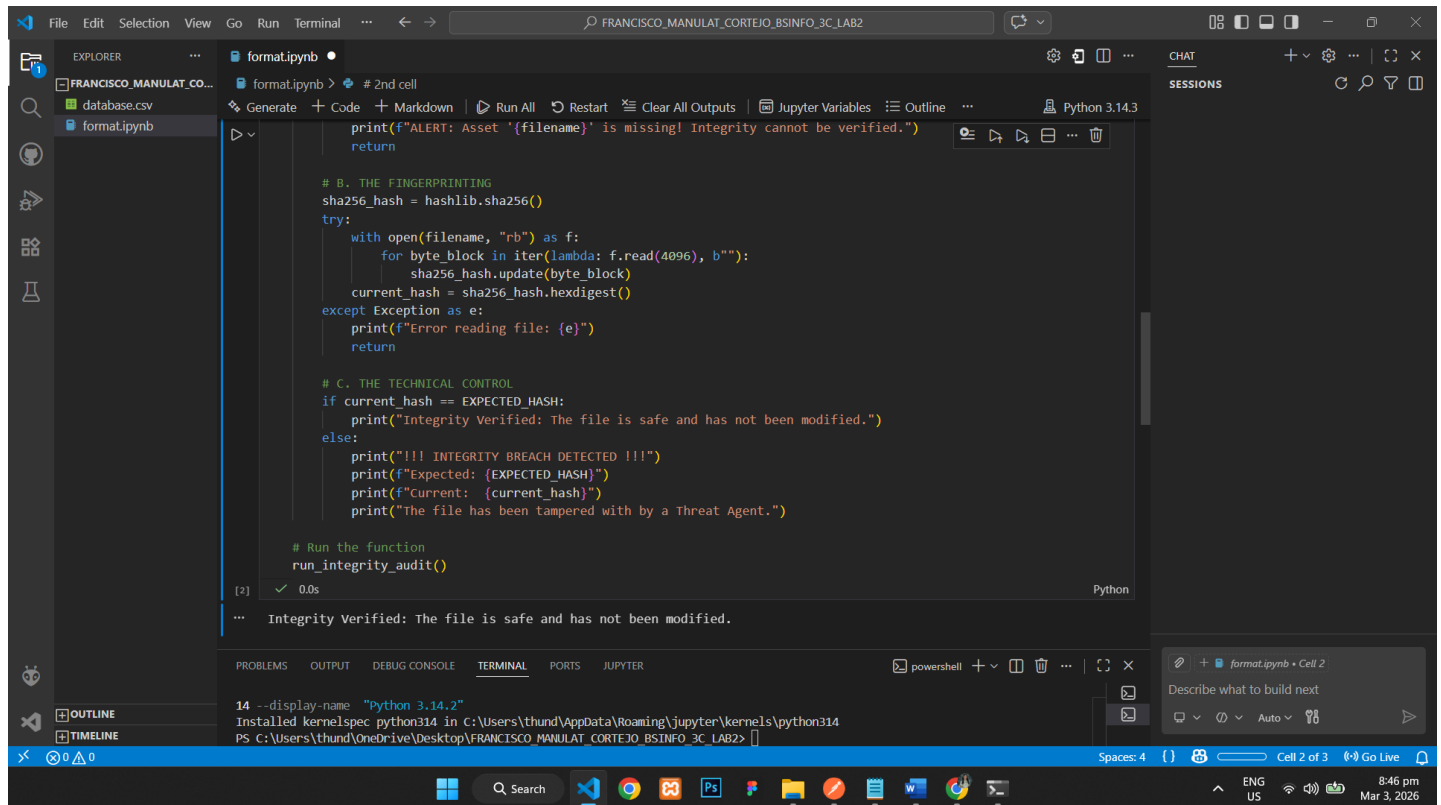
INFOE 3

LAB 2: HASHING

Francisco, Maica Merylle

Manulat, Faith Joy

Cortejo, Kevin Edyson



The screenshot shows a Jupyter Notebook titled 'format.ipynb' in a VS Code editor. The notebook contains a Python script for file integrity auditing. The script defines a function `run_integrity_audit()` that calculates the SHA256 hash of a file and compares it to an expected hash. The output of the script is displayed in the terminal.

```
print(f"ALERT: Asset '{filename}' is missing! Integrity cannot be verified.")
return

# B. THE FINGERPRINTING
sha256_hash = hashlib.sha256()
try:
    with open(filename, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    current_hash = sha256_hash.hexdigest()
except Exception as e:
    print(f"Error reading file: {e}")
    return

# C. THE TECHNICAL CONTROL
if current_hash == EXPECTED_HASH:
    print("Integrity Verified: The file is safe and has not been modified.")
else:
    print("!!! INTEGRITY BREACH DETECTED !!!")
    print(f"Expected: {EXPECTED_HASH}")
    print(f"Current: {current_hash}")
    print("The file has been tampered with by a Threat Agent.")

# Run the function
run_integrity_audit()
```

The terminal output shows the following message:

```
Integrity Verified: The file is safe and has not been modified.
```

This output shows the auditor script running when the file has not been modified. The computed hash value matches the `EXPECTED_HASH`, confirming that the object's integrity remains intact and has not been compromised.

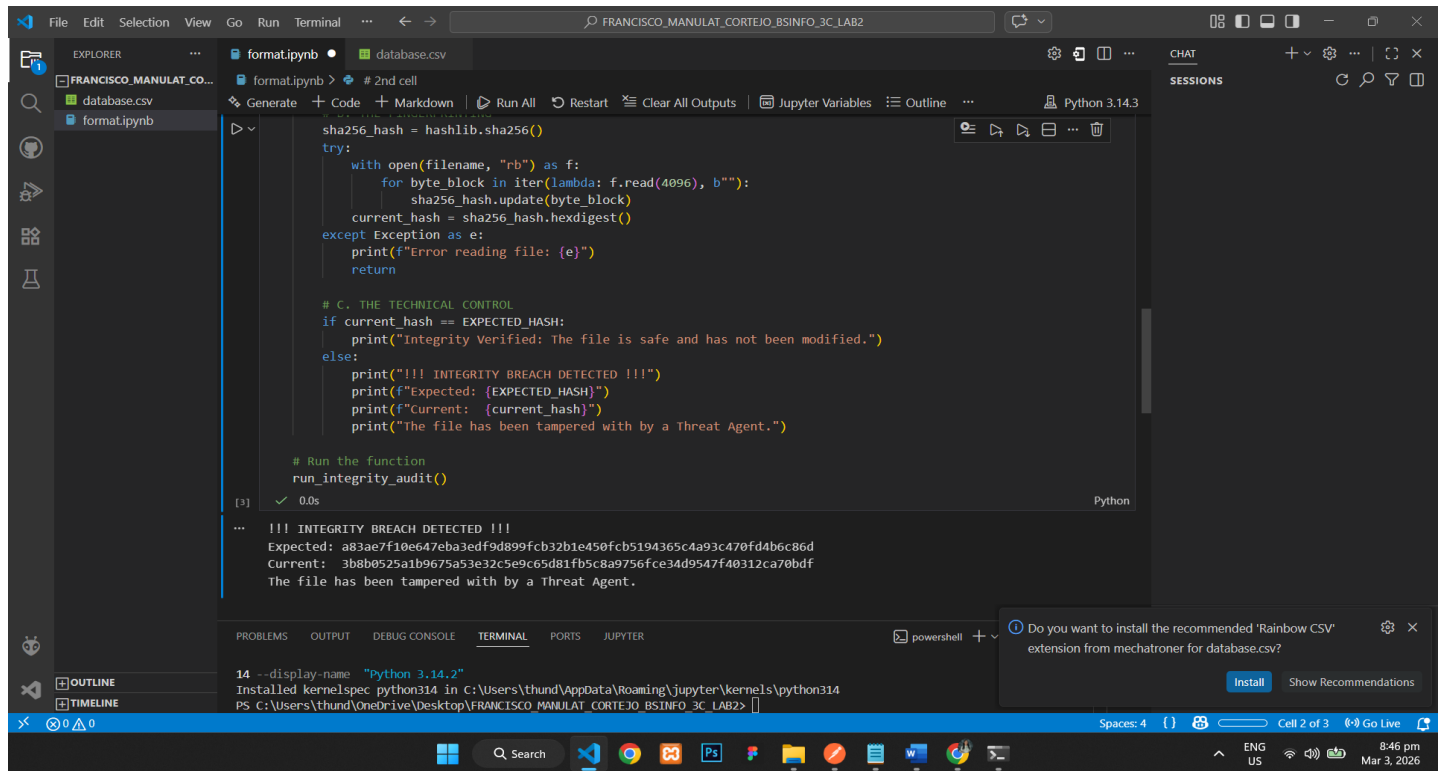
INFOE 3

LAB 2: HASHING

Francisco, Maica Merylle

Manulat, Faith Joy

Cortejo, Kevin Edyson



```
sha256_hash = hashlib.sha256()
try:
    with open(filename, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    current_hash = sha256_hash.hexdigest()
except Exception as e:
    print(f"Error reading file: {e}")
    return

# C. THE TECHNICAL CONTROL
if current_hash == EXPECTED_HASH:
    print("Integrity Verified: The file is safe and has not been modified.")
else:
    print("!!! INTEGRITY BREACH DETECTED !!!")
    print(f"Expected: {EXPECTED_HASH}")
    print(f"Current: {current_hash}")
    print("The file has been tampered with by a Threat Agent.")

# Run the function
run_integrity_audit()
```

```
!!! INTEGRITY BREACH DETECTED !!!
Expected: a83ae7f10e647eba3edf9d899fcb32b1e450fcb5194365c4a93c470fd4b6c86d
Current: 3b8b0525a1b9675a53e32c5e9c65d81fb5c8a9756fce34d9547f40312ca70bdf
The file has been tampered with by a Threat Agent.
```

14 --display-name "Python 3.14.2"
Installed kernelspec python314 in C:\Users\thund\AppData\Roaming\jupyter\kernels\python314
PS C:\Users\thund\OneDrive\Desktop\FRANCISCO_MANULAT_CORTEJO_BSINFO_3C_LAB2>

This screenshot shows the auditor activating a high-priority alert. Since the file was altered, the newly computed hash value no longer matches the original protection profile, which successfully indicates that the file's integrity has been compromised.